



UNIVERSITY *of* WEST FLORIDA

Machine Learning with Scikit-learn

Recurrent Neural Networks (RNN)

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Overview

Recurrent Neural Networks (RNNs)

Overview

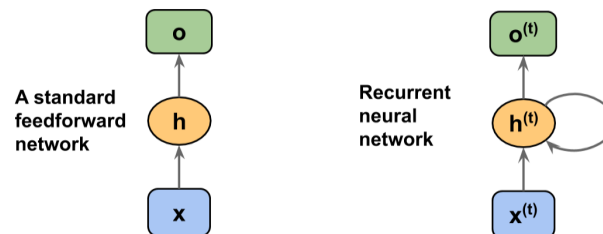
Recurrent Neural Networks (RNNs) are a class of neural networks designed for sequential data. Unlike traditional feedforward networks, RNNs have connections that allow information to persist, making them suitable for tasks involving time series, natural language processing, and speech recognition.

Key Features

- **Sequential Processing:** Maintains memory of previous inputs through hidden states.
- **Shared Weights:** Same parameters are used for each time step, reducing complexity.
- **Backpropagation Through Time (BPTT):** A variant of backpropagation used to train RNNs.

Challenges

- **Vanishing Gradient Problem:** Gradients shrink over long sequences, making learning difficult.
- **Exploding Gradient Problem:** Large gradients cause unstable updates.
- **Short-Term Memory:** Difficulty in capturing long-range dependencies.

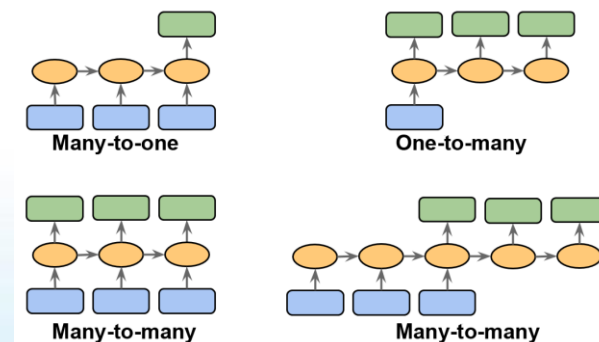
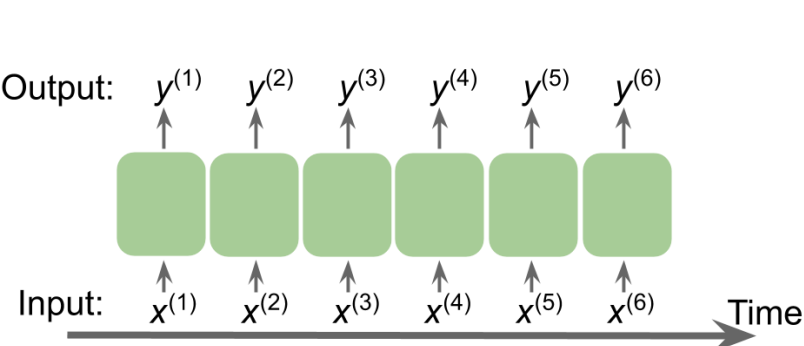


Sequential Data

Sequential data, e.g., time-series data, DNA sequences, NLP.

The most common sequencing tasks:

- **Many-to-one:** The input data is a sequence, but the output is a fixed-size vector or scalar, not a sequence. For example, in sentiment analysis, the input is text-based (for example, a movie review) and the output is a class label (for example, a label denoting whether a reviewer liked the movie).
- **One-to-many:** The input data is in standard format and not a sequence, but the output is a sequence. An example of this category is image captioning—the input is an image and the output is an English phrase summarizing the content of that image.
- **Many-to-many:** Both the input and output arrays are sequences. This category can be further divided based on whether the input and output are synchronized. An example of a synchronized many-to-many modeling task is video classification, where each frame in a video is labeled. An example of a delayed many-to-many modeling task would be translating one language into another. For instance, an entire English sentence must be read and processed by a machine before its translation into German is produced.

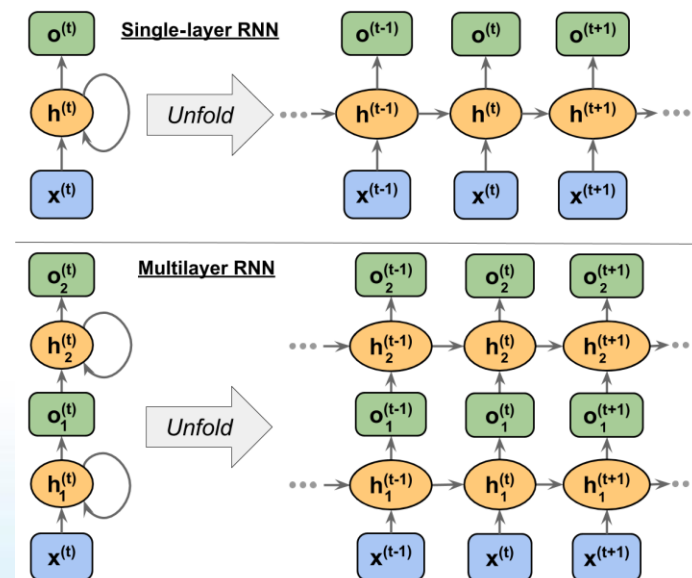
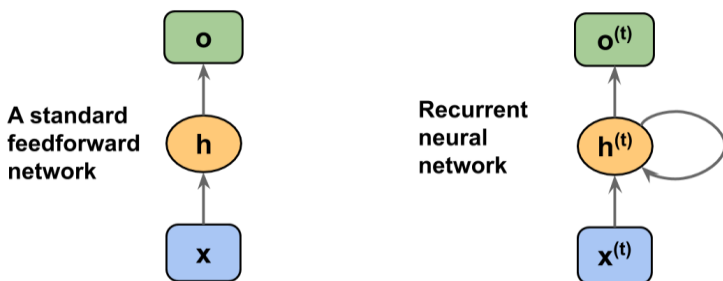


RNN Data Flow

Assume that the input layer (x), hidden layer (h), and output layer (o) are vectors that contain many units, where the input sequence is usually expressed as $\langle \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T)} \rangle$.

The flow of information (a.k.a. **recurrent edge**) in adjacent time steps in the hidden layer allows the network to have a memory of past events.

It's a common convention to refer to RNNs with one hidden layer as a *single-layer RNN*, which is not to be confused with single-layer NNs without a hidden layer, such as Adaline or logistic regression.



RNN Data Flow

At the first time step, $t = 0$, the hidden units are initialized to zeros or small random values. Then, at a time step where $t > 0$, the hidden units receive their input from

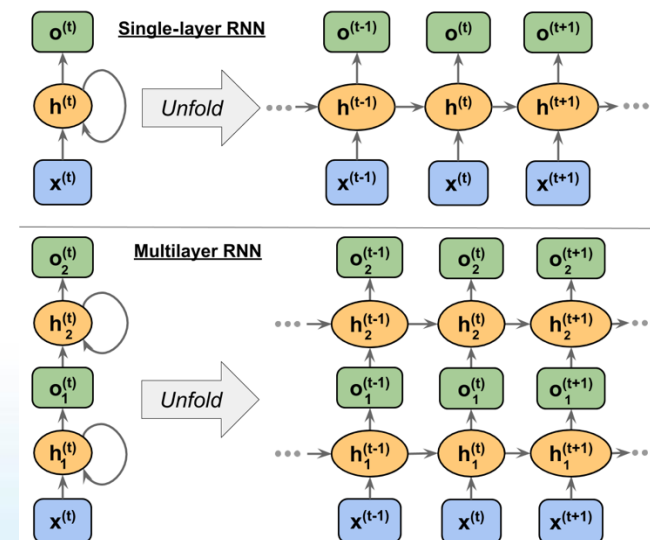
1. the data point at the current time, $x^{(t)}$, and
2. the previous values of hidden units at $t - 1$, indicated as $h^{(t-1)}$.

Similarly, in the case of a multilayer RNN, we can summarize the information flow:

- layer = 1 : Here, the hidden layer is represented as $h_1^{(t)}$ and it receives its input from the data point, $x^{(t)}$, and the hidden values in the same layer, but at the previous time step, $h_1^{(t-1)}$.
- layer = 2: The second hidden layer, $h_2^{(t)}$, receives its inputs from the outputs of the layer below at the current time step ($o_1^{(t)}$) and its own hidden values from the previous time step, $h_2^{(t-1)}$.

Since, in this case, each recurrent layer must receive a sequence as input, all the recurrent layers except the last one must return a sequence as output.

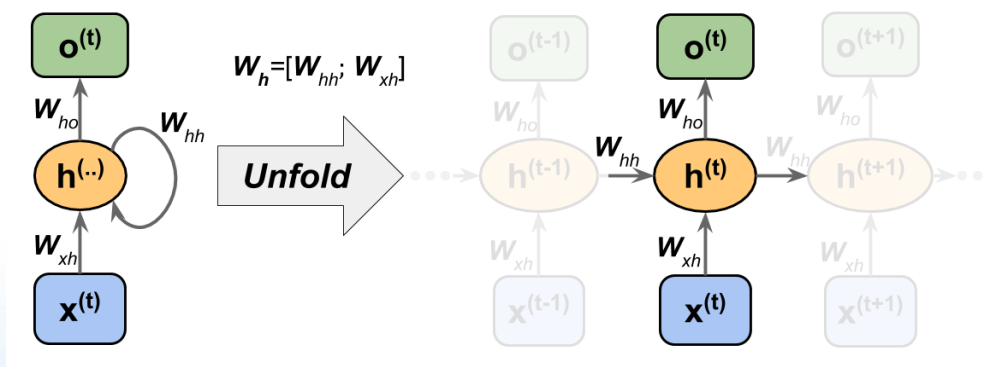
The behavior of the last recurrent layer depends on the type of problem.



RNN Activations

Let's consider just a single hidden layer RNN (same concept applies to multilayer RNNs.) Each directed edge (the connections between boxes) in the representation of an RNN that we just looked at is associated with a weight matrix. Those weights do not depend on time, t .

- W_{xh} : The weight matrix between the input, $x^{(t)}$, and the hidden layer, h
- W_{hh} : The weight matrix associated with the recurrent edge
- W_{ho} : The weight matrix between the hidden layer and output layer



RNN Activations

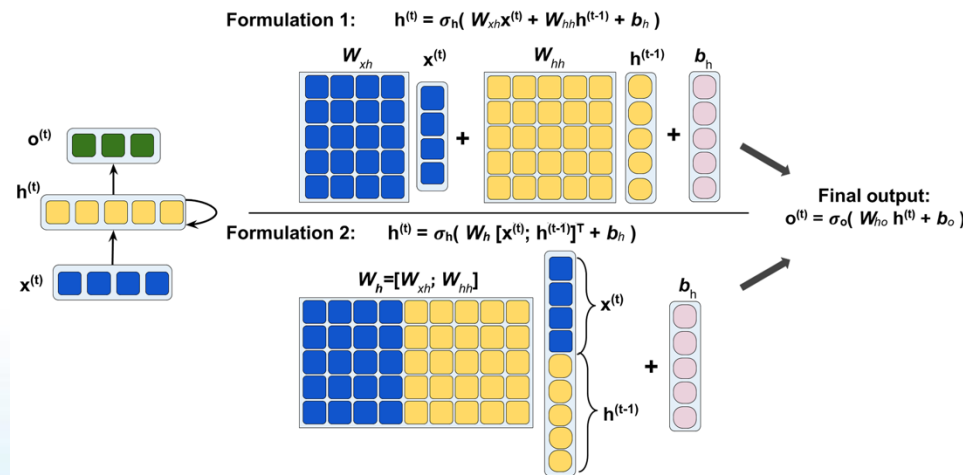
Similar to standard multilayer perceptrons and other types of feedforward NNs. For the hidden layer, the net input, z_h (preactivation), is computed through a linear combination:

$$\mathbf{z}_h^{(t)} = \mathbf{W}_{zh} \mathbf{x}^{(t)} + \mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{b}_h$$

Then, the activations of the hidden units at the time step, t , are calculated as follows:

$$\mathbf{h}^{(t)} = \sigma_h \left(\mathbf{z}_h^{(t)} \right) = \sigma_h \left(\mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{b}_h \right)$$

- $\mathbf{b}_h$ is the bias vector for the hidden units
- $\sigma(\cdot)$ is the activation function of the hidden layer.



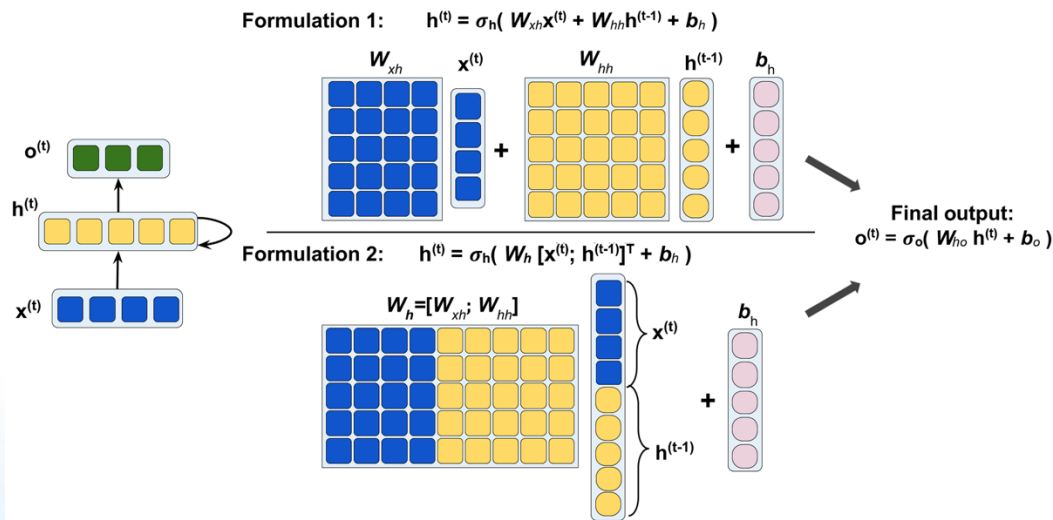
RNN Activations

In certain implementations, the weight matrices, W_{xh} and W_{hh} , are concatenated to a combined matrix, $W_h = [W_{xh}; W_{hh}]$, then the formula for computing hidden units will change, as follows:

$$\mathbf{h}^{(t)} = \sigma_h \left([W_{zh}; W_{hh}] \begin{bmatrix} \mathbf{x}^{(t)} \\ \mathbf{h}^{(t-1)} \end{bmatrix} + \mathbf{b}_h \right)$$

Once the activations of the hidden units at the current time step are computed, then the activations of the output units will be computed, as follows:

$$\mathbf{o}^{(t)} = \sigma_o (W_{ho} \mathbf{h}^{(t)} + \mathbf{b}_o)$$



Hidden recurrence versus output recurrence

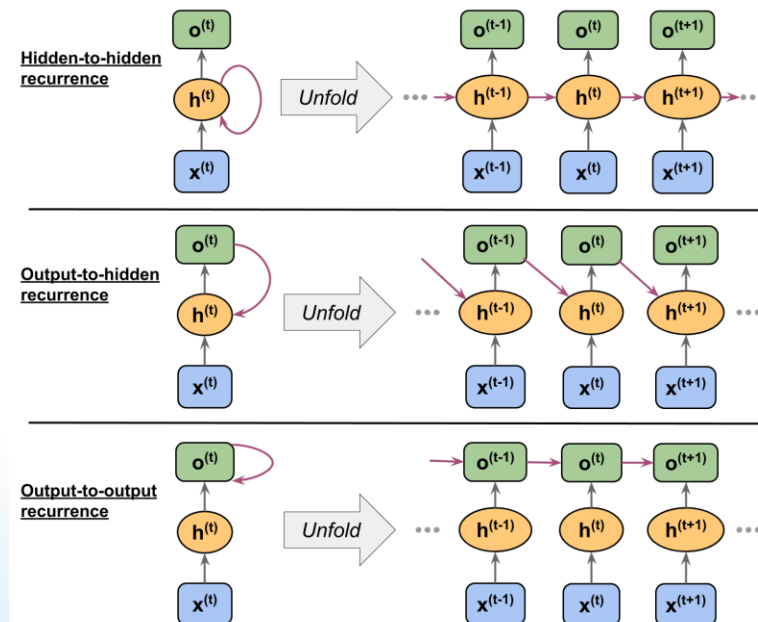
So far, you have seen recurrent networks in which the hidden layer has the recurrent property. (hidden-to-hidden recurrence)

There are different models in which the recurrent connection comes from the output layer. The net activations from the output layer at the previous time step, $\mathbf{o}^{t-1}$, can be added in one of two ways:

- To the hidden layer at the current time step, $\mathbf{h}^t$ (output-to-hidden recurrence)
- To the output layer at the current time step, $\mathbf{o}^t$ (output-to-output recurrence)

The weights associated with the recurrent connection will be denoted for the

- hidden-to-hidden recurrence by $\mathbf{W}_{hh}$,
 - output-to-hidden recurrence by $\mathbf{W}_{oh}$,
 - output-to-output recurrence by $\mathbf{W}_{oo}$.
- (In some articles in literature, the weights associated with the recurrent connections are also denoted by $\mathbf{W}_{rec}$.)



Long Short-Term Memory (LSTM)

Training RNNs using backpropagation through time (BPTT)

The derivation of the gradients might be a bit complicated, but the basic idea is that the overall loss, L , is the sum of all the loss functions at times $t = 1$ to $t = T$:

$$L = \sum_{t=1}^T L^{(t)}$$

Since the loss at time t is dependent on the hidden units at all previous time steps $1:t$, the gradient will be computed as follows:

$$\frac{\partial L^{(t)}}{\partial \mathbf{W}_{hh}} = \frac{\partial L^{(t)}}{\partial \mathbf{o}^{(t)}} \times \frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}} \times \left(\sum_{k=1}^t \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} \times \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{W}_{hh}} \right)$$

Here, $\frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(k)}}$ is computed as a multiplication of adjacent time steps:

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(i-1)}}$$

Long Short-Term Memory (LSTM)

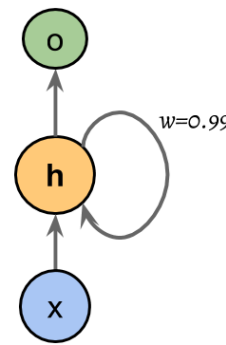
RNN BPTT issues

However, because of the multiplicative factor, $\frac{\partial h^{(t)}}{\partial h^{(k)}}$, in computing the gradients of a loss function, the so-called vanishing and exploding gradient problems arise.

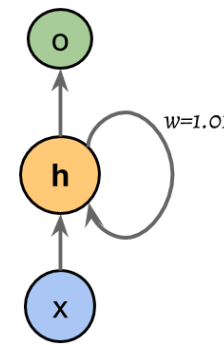
Basically, $\frac{\partial h^{(t)}}{\partial h^{(k)}}$ has $t - k$ multiplications; therefore, multiplying the weight, w , by itself $t - k$ times results in a factor, w^{t-k} . As a result, if the weight of the recurrent edge is

- $|w| < 1$, this factor becomes very small when $t - k$ is large.
- $|w| > 1$, then w^{t-k} becomes very large when $t - k$ is large. Note that a large $t - k$ refers to long-range dependencies. We can see that a naive solution to avoid vanishing or exploding gradients can be reached by ensuring $|w| = 1$.

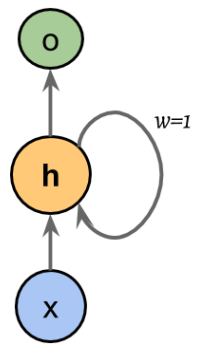
Vanishing gradient: $|w_{hh}| < 1$



Exploding gradient: $|w_{hh}| > 1$



Desirable: $|w_{hh}| = 1$



Long Short-Term Memory (LSTM)

In practice, there are at least three solutions to this problem:

- **Gradient clipping:** We specify a cut-off or threshold value for the gradients, and we assign this cut-off value to gradient values that exceed this value.
- **Truncated backpropagation through time (TBPTT):** It simply limits the number of time steps that the signal can backpropagate after each forward pass. For example, even if the sequence has 100 elements or steps, we may only backpropagate the most recent 20 time steps.
- **LSTM:** It is more successful in vanishing and exploding gradient problems while modeling long-range dependencies through the use of memory cells.

Long Short-Term Memory (LSTM)

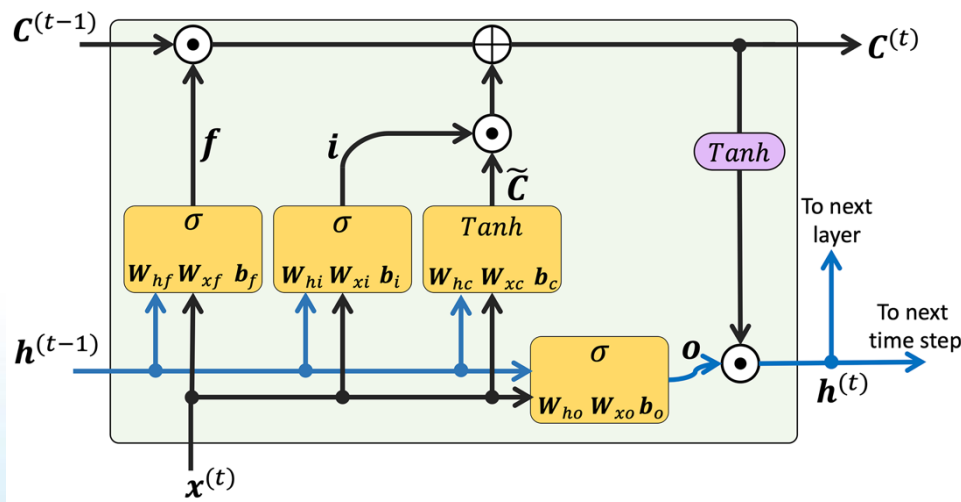
Memory cells

Definition

- The memory cell replaces or represents the hidden layer of standard RNNs.
- It includes a recurrent edge with a fixed weight $w = 1$, designed to address vanishing and exploding gradient problems.

Cell State ($\mathcal{C}^{(t)}$)

- The cell state refers to the values associated with the recurrent edge.
- It evolves from the previous time step's cell state ($\mathcal{C}^{(t-1)}$) to the current one ($\mathcal{C}^{(t)}$).
- This update occurs without direct multiplication by any weight factor.



Long Short-Term Memory (LSTM)

Information Flow Control

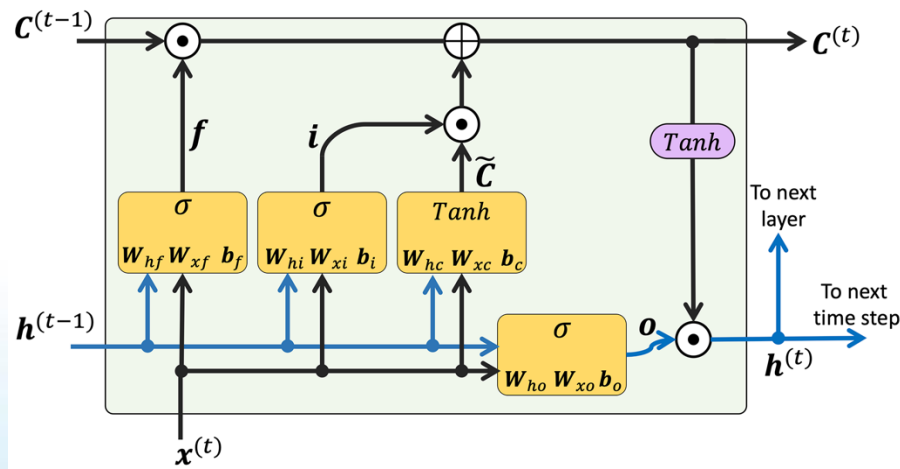
- Governed by several computational units known as gates.
- Operators:
 - $\odot$: Element-wise multiplication (product).
 - $\oplus$: Element-wise summation (addition).

Inputs and Hidden Units

- $x^{(t)}$: Input data at time step t .
- $h^{(t-1)}$: Hidden state at time step $t - 1$.

Computation Units

- Four computation units apply activation functions (sigmoid σ or tanh).
- These units perform matrix-vector multiplications on inputs $x^{(t)}$ and $h^{(t-1)}$.
- Units with sigmoid activation functions that pass output through $\odot$ are known as gates.



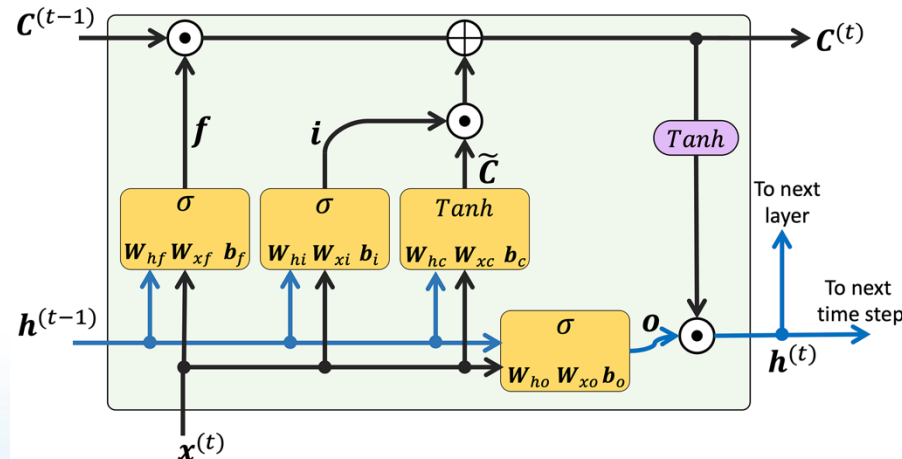
Long Short-Term Memory (LSTM)

Gates

Forget gate (f_t)

It allows the memory cell to reset the cell state without growing indefinitely. In fact, the forget gate decides which information is allowed to go through and which information to suppress.

$$f_t = \sigma(W_{xf}x^{(t)} + W_{hf}h^{(t-1)} + b_f)$$



Long Short-Term Memory (LSTM)

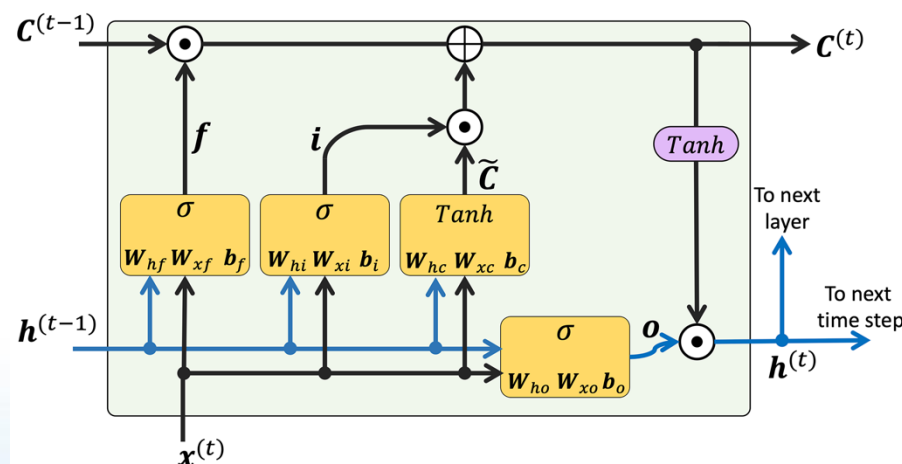
Input gate (i_t)

The input gate (i_t) and candidate value ($\tilde{c}_t$) are responsible for updating the cell state.

$$\begin{aligned} i_t &= \sigma(W_{xi}x^{(t)} + W_{hi}h^{(t-1)} + b_i) \\ \tilde{c}_t &= \tanh(W_{xc}x^{(t)} + W_{hc}h^{(t-1)} + b_c) \end{aligned}$$

The cell state at time t is computed as follows:

$$c^{(t)} = (c^{(t-1)} \odot f_t) \oplus (i_t \odot \tilde{c}_t)$$

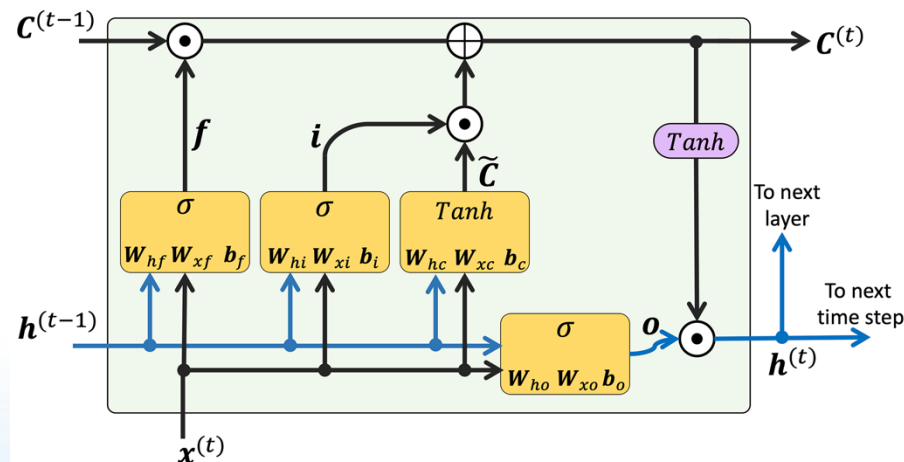


Long Short-Term Memory (LSTM)

Output gate (o_t)

The output gate (o_t) decides how to update the values of hidden units, $\mathbf{h}^{(t)}$.

$$\begin{aligned} o_t &= \sigma(W_{xo}x^{(t)} + W_{ho}h^{(t-1)} + b_o) \\ h^{(t)} &= o_t \odot \tanh(C^{(t)}) \end{aligned}$$



Go to Lecture Code

Let's Implement RNN